面试评估报告

智能体开发工程师面试评估报告

总体评估

- **面试结果：** ✅**通过**
- **总题数：** 4 题
- **合格题数：** 4 题 (要求 2 题及格)
- **不合格题数：** 0 题

面试总结与建议

{采访录音}

问答明细

第 1 题：好的，了解了你的技术背景。那么接下来，请用 Python 写一个简单的 RAG 搜索函数，输入用户查询和文档列表，返回最相关的文档片段。

你的回答：

输入：用户查询 + 文档列表
输出：最相关的文档带向量
嵌入 + 余弦相似度（RAG 标准方案）
无外部复杂依赖，只用 scikit-learn 做向量化
完整代码
python
运行
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def simple_rag_search(query: str, Documents: list[str]) -> str:
"""
简单RAG检索函数：输入用户问题和文档列表，返回最相关的文档片段
:param query:用户查询语句
:param文档:文档列表（字符串数组）
:return:最相似的文档
"""
```

if not Documents:
return "无可用文档"

```
    # 1. 把查询 + 文档合并，统一向量化
    texts = [query] + documents

    # 2. TF-IDF 向量化（轻量、无需模型、面试标准写法）
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(texts)

    # 3. 计算 query 与所有文档的余弦相似度
    query_vec = tfidf_matrix[0:1]  # 第一个是 query
    doc_vecs = tfidf_matrix[1:]    # 后面是文档
    similarities = cosine_similarity(query_vec, doc_vecs)[0]

    # 4. 找到最相似的文档
    best_index = similarities.argmax()
    best_doc = documents[best_index]

    return best_doc
```

---------- 测试示例 ----------

if **name** == " **main** ":
# 模拟知识库文档
docs = [
"智能体（Agent）是能够自主感知环境并执行任务的AI系统",
"RAG是检索增强生成，用于从知识库中检索相关信息",
"Python是一种解释型、高级编程语言",
"大模型通过海量数据训练能力，具备理解与生成"
]

```
    # 用户查询
    user_query = "什么是RAG？"

    # RAG 检索
    result = simple_rag_search(user_query, docs)
    print("最相关文档：", result)
```

面试官评价（✅合格）：

代码完整，逻辑清晰，正确实现TF-IDF支持化和余弦相似度计算，符合RAG基础要求。

第2题：好的，接下来请用Python实现一个智能体对话系统中的意图识别函数，输入用户消息，返回意图分类结果。

你的回答：

基于LLM最终构造输出的标准识别函数（智能体系统方案），直接对接硅基流动/任何大模型API，返回稳定的JSON格式。
版本：智能体系统识别函数（生产级）
python
runningimport
json
import requests

配置：替换成你的硅基流动API密钥

API_KEY = "sk-xxxxxxxxxxxxxxxxxxxxxxxxx"
MODEL_NAME = "Qwen2.5-72B-Instruct" # 硅基流动大模型
API_URL = " https://api.siliconflow.cn/v1/chat/completions "

def agent_intent_recognition(user_message: str) -> dict:
"""
智能体对话系统 - 意图识别函数
输入：用户消息
输出：{
"intent": 意图标签,
"confidence": 0-1 置信度,
"description": 意图说明
}
支持:、预约、投诉、感谢、闲聊、取消、难民、未知
"""
# 意图定义（智能体标准意图）
intent_list = [
"查询", "预约", "投诉", "感谢", "闲聊", "取消", "退款", "未知原因"
]

```
# 系统提示词（强制输出 JSON，Agent 必备）
system_prompt = f"""
你是智能体对话系统的意图识别引擎，严格按要求输出JSON，不添加任何额外文字。
只能从以下列表选择一个意图：{intent_list}

输出格式（严格JSON）：
{{
    "intent": "选中的意图",
    "confidence": 0.95,
    "description": "意图简单说明"
}}
"""

# 调用大模型 API
headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

data = {
    "model": MODEL_NAME,
    "temperature": 0.1,  # 低温度=更稳定
```

```
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message}
        ],
        "response_format": {"type": "none"}
    }

    try:
        response = requests.post(API_URL, headers=headers, json=data, timeout=10)
        response.raise_for_status()
        result = response.json()
        llm_output = result["choices"][0]["message"]["content"].strip()

        # 解析 JSON（结构化输出，智能体核心）
        intent_result = json.loads(llm_output)
        return intent_result

    except Exception as e:
        return {
            "intent": "未知意图",
            "confidence": 0.0,
            "description": f"识别异常：{str(e)}"
        }
```

---------- 测试 ----------

if **name** == " **main** ":
test_msgs = [
"我的订单查一下",
"帮我预约明天上门",
"我要投诉你们",
"谢谢",
"你好",
"我要取消订单",
"我要退款",
"宇宙起源是什么"
]

```
    for msg in test_msgs:
        res = agent_intent_recognition(msg)
        print(f"用户：{msg}")
        print(f"意图结果：{res}\n")
```

面试官评价（✅合格）：

> 回答完整展示了基于LLM的集成电路识别实现，包含API调用、错误处理、测试用例，符合生产级要求。

第3题：好的，接下来请谈谈在智能体系统中如何设计对话状态管理模块来跟踪多轮对话的上下文？

你的回答：

一、模块定位
对话状态管理（对话状态管理器，DSM）是智能体的「短期记忆 + 控制中心」：
记住用户是谁、聊到哪一步
记录当前任务、槽位、意图、历史
下一步：决定追问 / 调用工具 / 回答 / 结束语
总结：DSM = 上下文存储 + 状态机 + 槽位管理 + 历史管理 + 策略决策
二、数据结构（必须会写）
这是实际代码里的 DialogState，面试直接写：
python从数据类
运行
导入数据类、
输入字段导入列表、字典、可选

@dataclass
class DialogState:
# 会话唯一标识
session_id: str
user_id: str

```
# 意图（当前用户目标）
intent: Optional[str] = None
intent_confidence: float = 0.0

# 槽位（任务参数，如：时间、地点、订单号）
slots: Dict[str, Optional[str]] = field(default_factory=dict)
slot_status: Dict[str, str] = field(default_factory=dict)  # filled / missing

# 多轮历史
history: List[Dict[str, str]] = field(default_factory=list)  # user / assistant

# 系统状态
state: str = "idle"       # idle / collecting / thinking / tool_calling / finished
turn: int = 0             # 轮次
last_action: str = ""     # 上一步动作
context: Dict = field(default_factory=dict)  # 扩展上下文
```

作用：
意图：当前任务目标
槽位：任务所需参数（智能体 = 意图 + 槽位）
状态：状态机控制对话走向
历史：多轮问上下文
三、流程状态机设计（标准 5 状态）
智能体通用状态流转：
空闲：空闲，等待输入用户
理解：意图识别 + 槽位抽取
收集：放弃槽位，主动追
执行：调用工具 / API / 知识库
响应：生成回答

完成：任务完成/退出
典型：
空闲 → 理解→ 收集 → 执行 → 响应 → 完成
四、上下文管理策略（重点）

1. 截断历史断策略
   保留最近N轮（如5~10轮）
   超过长度：摘要压缩+保留关键槽位
   防止LLM下游溢出
2. 上下文分层（高级）
   短期记忆：当前会话（状态+历史）
   中期记忆：用户画像、偏好
   长期记忆：RAG 知识库
3. 槽位优先级
   高优槽位优先追问（如预约必须要：时间 + 服务）
   自动去重、覆盖、校验
   五、核心模块接口（代码级设计）
   python
   运行
   class DialogStateManager:
   def **init** (self):
   self.sessions: Dict[str, DialogState] = {}

1.创建/获取会话

def get_or_create_state(self, session_id: str, user_id: str) -> DialogState:
...

2.更新意图

def update_intent(self, session_id: str, intent: str, confidence: float):
...

3.更新槽位（核心）

def update_slots(self, session_id: str, slots: Dict[str, str]):
...

4.检查槽位

```
def get_missing_slots(self, session_id: str) -> List[str]:
...
```

5.追加对话历史

```
def add_turn(self, session_id: str, user_msg: str, assistant_msg: str):
...
```

6. 状态流转

```
def transition_state(self, session_id: str, new_state: str):
...
```

7. 清空会话

```
def clear_state(self, session_id: str):
...
```

六、多轮对话示例流程
用户：我想预约上门服务→ DSM：intent = 预约，slots={}，返回：时间、地址→ DSM 状态 → 领取→ 代理：请问您想预约什么时间?
用户：明天下午 3 点→ DSM：slots={"time":" 明天 15:00"}→ 尚缺地址→ 代理：请问你的地址是吗?
用户：北京市朝阳区...→ slot 填满→ 状态 →正在执行→调用预约工具→返回成功→状态→完成→代理：预约完成!

面试官评价（✅合格）:

回答详实，涵盖数据结构、状态机、管理策略和代码实现，技术深度满足岗位要求。

第4题：好，那么接下来请谈谈在多智能体协作场景中，如何设计消息路由机制来避免循环调用?

你的回答:

多智能体消息路由要避免循环调用（死循环、乒乓转发），核心思路就4条：
消息带唯一ID + 路由路径（Trace）
设置最大转发次数（Hop Limit）
明确目标地址，不广播、不泛洪去
状态检测：已处理消息不二次调度
下面我按设计方案→数据结构→路由逻辑→防循环策略→伪代码讲完整。
一、整体架构：中央路由+点对点通信

避免循环的最佳做法不是P2P网络状，相反：中央路由（Router）+ 注册制 Agent + 显式跳目标结构：
Agent 只和 Router 通信
Agent 之间不直接互发消息
Router 统一调度、去重、验权、限优点：从源头杜绝A
→B→A→B…这种循环。
二、消息结构设计（关键！）每条必须
进行路由头，这是防消息循环基础。

```
@dataclass
class AgentMessage:
# 唯一标识
msg_id: str = uuid.uuid4().hex
# 发送方
sender: str
# 接收目标接收方（式显指定）
receiver: str
#
content: dict
# 路由路径（经过哪些Agent）
trace: List[str] = None
# 跳内容当前跳数
: int = 0
# 最大允许转发次数（防循环）
max_hop: int = 3
# 消息类型
type: str = "request" # 请求/响应/广播
```

关键点：
trace记录路径：A → B → C
跳计数
max_hop硬上限
接收器必须明确，不能泛洪
三、防循环核心机制（4条）
1）最大跳数限制（TTL机制）
每转发一次hop+1，超过max_hop直接转发。
明文
hop >= max_hop → 拒绝转发
通常设置3~5足够，任务型协作不会太深。
2）路由轨迹检测（LoopDetection）
如果发送者已经在踪迹里 → 判定循环。
plaintext

if sender in trace → 拒绝转发
直接杜绝：A→B→A→B...
3）消息去重（Message Deduplication）
Router 维护近期 msg_id 集合：
plaintext
if msg_id inprocessed → 丢弃
防止重复调度、重入。
4）强制请求 / 应答转发（Request-Response）
请求：A → B
响应：B → A（不能再转发回 B）
Response类型消息禁止再次路由，从逻辑上断环。
四、路由算法伪代码（可直接写白板）
python
运行
class AgentRouter:
def **init** (self):
self.agents = {} # 注册的智能体
self.processed = set() # 已处理消息ID
self.max_trace = 100

```
def send_message(self, msg: AgentMessage):
    # 1. 去重
    if msg.msg_id in self.processed:
        return "duplicate"
    self.processed.add(msg.msg_id)

    # 2. 跳数超限
    if msg.hop >= msg.max_hop:
        return "hop limit exceeded"

    # 3. 路径环检测
    if msg.sender in msg.trace:
        return "loop detected: sender in trace"

    # 4. 添加轨迹
    msg.trace.append(msg.sender)
    msg.hop += 1

    # 5. 转发到目标Agent
    if msg.receiver not in self.agents:
        return "receiver not exist"

    agent = self.agents[msg.receiver]
    return agent.on_receive(msg)
```

Agent接收逻辑
python
运行
class BaseAgent:
def **init** (self, name, router):
self.name = name
self.router = router

```
def send(self, receiver, content):
    msg = AgentMessage(
        sender=self.name,
        receiver=receiver,
        content=content
    )
    return self.router.send_message(msg)

def on_receive(self, msg):
    # 处理逻辑...
    # 回复时必须指定 receiver=msg.sender
    # 不会产生循环
    self.send(msg.sender, {"result": "ok"})
```

五、为什么这样不会循环？（必讲）
不广播，必须指定接收方 → 不会乱转发
trace 记录路径 → 回归路直接封杀
上限 → 深度有限
请求/响应拦截 → 回复只能回到上游，不能继续扩散
路由器中心化 → 所有消息统一验证
A→B→A 会被跟踪检测 直接拦截A→B→C→B→A 会被跳数限制
六拦截、进阶策略（可加分）任务
ID隔离：同任务内消息才转发
权限白名单：谁能转发、谁只能回复
超时自动终止：超过时间不再路由
有向无环图 DAG 调度：严格按流程走，不允许逆向
LLM 决策约束：提示 强制禁止重复调用

面试官评价（✅合格）：

回答具体，架构清晰，防循环机制全面，伪代码具体安装。